

TrafficVision AI

Milestone 1 Preparatory Report — Understanding, Design & Research

1. Understanding the Problem Statement

TrafficVision AI is an AI-powered traffic prediction and congestion management platform built for city authorities, traffic control departments, transportation agencies, and smart-city initiatives.

The platform must solve four connected problems:

- City authorities have no single, real-time view of traffic conditions across the city.
- Congestion is discovered reactively, after it has already built up, instead of being predicted in advance.
- Commuters and operators have no reliable way to receive alternate-route or delay guidance during congestion.
- Historical traffic patterns are not analyzed systematically, so long-term planning decisions lack data support.

The solution is a centralized dashboard covering five functions: traffic monitoring, congestion prediction, route analysis, traffic analytics, and smart alerts — deployed on Docker and a cloud platform (AWS or Azure).

2. Objectives and User Roles

2.1 Core Objectives

- Monitor live traffic conditions (vehicle density, road utilization, congestion status) across all tracked locations.
- Predict congestion levels and peak-hour traffic using historical and real-time data.
- Recommend alternate routes and estimate travel time to reduce commuter delay.
- Notify the right people immediately when congestion, accidents, or road closures occur.
- Provide analytics dashboards (heatmaps, trend reports) that support long-term transportation planning.

2.2 User Roles

Attribute	Description
Admin	Manages users, configures sensors/data sources, and has full access to every module.
Traffic Operator	Monitors live dashboards, verifies alerts, and manages day-to-day congestion response.
Analyst / Planner	Uses analytics and historical reports for trend analysis and city planning decisions.
Public / Commuter	Views live traffic status, alerts, and route recommendations (read-only access).

3. Dashboard Research

Before wireframing, the following dashboard patterns from comparable traffic and monitoring systems should be reviewed:

- Google Maps Traffic Layer — color-coded live congestion overlay (green / amber / red).
- Waze Live Map — crowd-reported incidents and rerouting alerts on a live map.
- City-level smart traffic control dashboards (e.g. Singapore’s traffic management systems) — centralized monitoring with sensor-level drill-down.
- Grafana / Power BI style operations dashboards — for layout patterns: KPI cards on top, trend charts in the middle, detail tables below.

Key takeaways to apply: use a map-first layout for monitoring, colour-coded severity for alerts, and a separate analytics view for historical trends rather than mixing live and historical data on one screen.

4. UI Reference Collection

UI references to collect and shortlist before wireframing:

- Map-based live dashboards with sensor pins and colour-coded road segments.
- Admin panel layouts with a left-side navigation bar and role-based menu visibility.
- Alert/notification panels showing severity badges (e.g. “High”, “Medium”, “Low”).
- Analytics dashboard layouts using heatmaps, line charts, and summary KPI cards.
- Mobile-responsive traffic app layouts, for the public/commuter view.

These references should be organized into a shared UI moodboard (e.g. Figma or Canva) tagged by module: Monitoring, Prediction, Route Analysis, Alerts, Analytics.

5. Initial Wireframes to Use

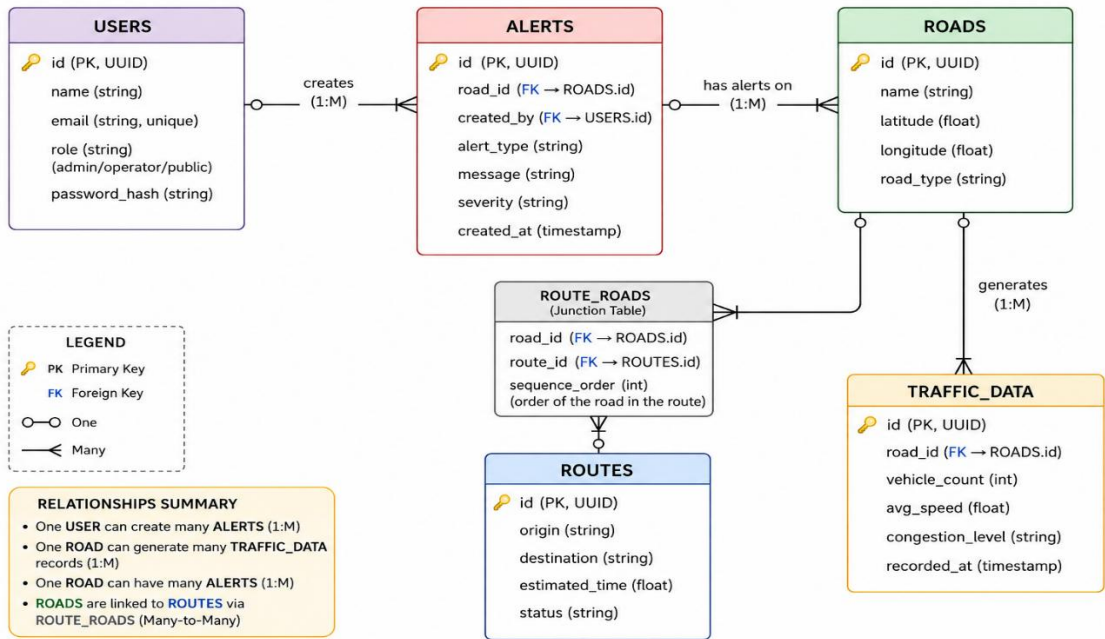
Recommended low-fidelity wireframes for the first design pass:

- Login / Role Selection screen — single entry point branching into Admin, Operator, Analyst, and Public views.
- Live Monitoring Dashboard — map view (left/centre) with a live status panel and KPI cards (right).
- Congestion Prediction View — forecast graph with a peak-hour timeline and confidence indicator.
- Route Analysis Screen — origin/destination input, suggested routes list, and travel-time comparison.
- Alerts & Notifications Panel — chronological feed with severity tags and filters.
- Analytics Dashboard — heatmap, trend charts, and a historical data filter (date range, location).

These six screens form the minimum wireframe set for Milestone 1 and should be reviewed before backend integration begins.

6. Draft ER Diagram

The entity-relationship diagram below reflects the core data entities implied by the architecture (User Management, Traffic Monitoring, Prediction, Route Analysis, Alerts, and Analytics services).



Core entities and key attributes:

USERS

- id (PK, UUID)
- name (string)
- email (string, unique)
- role (string — admin/operator/public)
- password_hash (string)

ALERTS

- id (PK, UUID)
- road_id (FK → ROADS.id)
- created_by (FK → USERS.id)
- alert_type (string)
- message (string)
- severity (string)
- created_at (timestamp)

ROADS

- id (PK, UUID)
- name (string)
- latitude (float)
- longitude (float)
- road_type (string)

TRAFFIC_DATA

- id (PK, UUID)
- road_id (FK → ROADS.id)
- vehicle_count (int)
- avg_speed (float)
- congestion_level (string)
- recorded_at (timestamp)

ROUTES

- id (PK, UUID)
- origin (string)
- destination (string)
- estimated_time (float)
- status (string)

ROUTE_ROADS (junction table, resolves the Roads ↔ Routes many-to-many)

- road_id (FK → ROADS.id)
- route_id (FK → ROUTES.id)
- sequence_order (int — order of the road within the route)

Relationships summary

- One USER creates many ALERTS (1:M)
- One ROAD generates many TRAFFIC_DATA records (1:M)
- One ROAD can have many ALERTS (1:M)
- ROADS and ROUTES are linked via ROUTE_ROADS (M:M)

7. Congestion Monitoring Research

Approaches to review before implementing the Traffic Monitoring and Prediction modules:

- Sensor-based monitoring — inductive loop detectors, CCTV with vehicle counting, and GPS/probe data from connected vehicles.
- Congestion-level classification — typically expressed as a ratio of current speed to free-flow speed, grouped into bands such as Free / Moderate / Heavy / Severe.
- Prediction techniques — time-series models (ARIMA, Prophet) for short-term forecasting, and machine-learning models (Random Forest, LSTM) for pattern-based peak-hour prediction.
- Public traffic APIs — Google Maps Traffic API, TomTom Traffic API, and OpenStreetMap-based routing engines, useful as reference data sources or fallbacks.
- Alerting thresholds — defining when a congestion level should automatically trigger a notification, to avoid alert fatigue from minor fluctuations.

This research should directly inform the congestion-level bands and prediction model choice used in the Traffic Prediction and AI Prediction modules (Milestone 2).